Cultural Mood Tracker ETL Report

Project repository summary based on the implemented ETL pipeline

1. Goal of the Project

The goal of this project is to build a reliable, modular, and reproducible data engineering pipeline for a Cultural Mood Tracker focused on recent English-language movies and TV shows. The implemented repository is not a generic media dataset collector. It is a deliberately scoped ETL system that starts with a one-year release window, extracts aligned data from multiple public sources, transforms that data into canonical structured tables, and prepares the resulting text and metadata for later retrieval-augmented generation use. The central idea is that cultural response to a title cannot be understood through one signal alone. A movie or show can be popular, critically discussed, actively searched, and heavily reviewed at the same time, and those signals do not always move together. The project therefore attempts to preserve those different perspectives while still organizing them around one consistent internal title identity.

In practical terms, the project is designed to answer the data engineering side of the assignment first. It establishes a stable extraction layer, a canonical transformation layer, validation and deduplication logic, run-based provenance, and an optional cloud publishing step. It does not yet implement the chatbot itself, but it intentionally structures the outputs so that a RAG system can be added later without needing to redesign the ETL foundation. The immediate objective is therefore to create a trustworthy data asset, not merely to scrape text. That asset should allow downstream analysis of title metadata, audience reception, editorial discussion, and public attention trends over time, while remaining clean enough to support SQL analytics and chunk-based retrieval later.

The scope has been intentionally narrowed to movies and TV shows, and to the most recent one-year window, because this creates a manageable and defensible baseline. The system uses TMDB as the anchor source for selecting the target universe of titles, and then enriches that anchor set using IMDb, TVMaze, Wikidata, Wikipedia Pageviews, Guardian coverage, and optionally GDELT. The outcome is a local-first ETL workflow that can run reproducibly, write raw data snapshots, produce processed canonical tables, emit validation reports, and optionally publish those outputs into Google Cloud Storage and BigQuery.

2. What Data Sources We Considered

During scoping, we considered a wider set of potential data sources than what was ultimately kept in the baseline implementation. The implemented repository now focuses on a practical subset that balances coverage, consistency, ease of matching, and usefulness for later retrieval. The baseline sources are TMDB, IMDb public datasets, TVMaze, Wikidata, Wikipedia Pageviews, Guardian Open Platform, and optional GDELT. We also discussed broader and noisier corpora such as Common Crawl, but did not include them in the implemented baseline because they would make the system substantially more fragile without improving the core data engineering story proportionally.

TMDB was selected as the anchor because it provides discover endpoints, title-level metadata, and review text in an accessible way. IMDb public datasets were included because they offer stable tabular metadata and rating information that can reinforce title identity and public rating signals. TVMaze was included because TV titles benefit from supplemental structured metadata that TMDB alone does not always express as cleanly for series-specific contexts. Wikidata was included because entity linking is

essential in any multi-source pipeline, and it provides a strong way to align external identities and find the corresponding English Wikipedia article. Wikipedia Pageviews was included because public attention is a different signal from audience sentiment or ratings, and the project needs a time-based attention layer. Guardian Open Platform was included because it contributes a more editorial, curated form of text evidence than audience reviews, which is valuable for later retrieval and grounded explanation. GDELT was included only as an optional source because, although it can provide broad media coverage, it is substantially noisier, more weakly aligned, and more operationally fragile.

Some sources were consciously excluded from the main implementation. Common Crawl, for example, was considered conceptually useful because it contains a huge amount of web text, but it is not a good baseline source for this project. It introduces uncontrolled heterogeneity, weak metadata, duplication, difficult entity linking, and a much larger document cleaning burden. Those issues would overwhelm the value of a well-scoped assignment pipeline. For similar reasons, the project did not expand into books or anime, because the metadata and cross-source matching foundations for those domains were weaker relative to the movie and TV baseline. The selected source set therefore reflects a design choice: prefer a smaller, alignable, monitorable corpus over a much larger but noisier one.

3. What the Data Sources Provide, What Makes Them Different and Important

Each source in the project contributes a distinct kind of information, and the value of the pipeline comes from combining those signals without flattening their differences. TMDB is the operational backbone of the project. It is the source used to discover the target titles in the configured release window, and it provides title metadata such as names, release dates, genres, original language, popularity, vote averages, vote counts, overviews, and review counts. Most importantly, it also provides user review text, which gives the project direct access to audience-written natural language. This makes TMDB uniquely valuable because it supplies both the structured metadata needed to define the title universe and the unstructured review content needed for later text retrieval.

IMDb serves a different role. It is not used here as the anchor discovery source, but as a stabilizing enrichment source through its public TSV datasets. The repository uses matched subsets of `title.basics.tsv.gz` and `title.ratings.tsv.gz`. These files contribute title types, primary and original titles, runtime, genres, start year, end year, average rating, and vote count. IMDb matters because it provides a second, widely recognized reception signal that can be compared against TMDB ratings, and because it reinforces entity resolution through IMDb IDs. It is therefore less important as a text source and more important as a normalization and validation source.

TVMaze is narrower but strategically useful. It primarily enriches TV titles through fields such as language, status, genres, network, and summary text. Its TV-centric orientation means it helps the project avoid treating TV as a weaker version of movies. It gives supplemental series metadata and summary text that can later become retrieval evidence or at least descriptive context.

Wikidata is the project’s entity alignment backbone. It provides stable entity IDs, English labels, English descriptions, and most importantly the mapping to English Wikipedia article titles. This matters because a multi-source ETL system fails quickly if it cannot confidently determine whether different sources are talking about the same title. Wikidata improves that situation by acting as a bridge between title identity and external knowledge references.

Wikipedia Pageviews provides a completely different kind of signal: not sentiment, not rating, and not editorial interpretation, but attention over time. A title can have a large number of pageviews because it is newly released, controversial, acclaimed, confusing, or simply popular. That ambiguity is precisely

what makes the source useful. It captures public information-seeking behavior and can later be compared against ratings and text sentiment to reveal gaps between visibility and reception.

Guardian Open Platform contributes editorial or journalistic text. This is different from TMDB user reviews because the language is typically more formal, more contextual, and more interpretive. Guardian content can support later chatbot answers that need contextual or media-style evidence rather than only fan or viewer reactions. It therefore improves the diversity of document types in the corpus and supports more grounded responses later.

GDELT, when enabled, contributes broad media references, but it is weaker than the other sources in quality and alignment. The project's code already treats it more cautiously, and it is disabled by default. Its main value is as an optional expansion path rather than a trusted core evidence layer. Taken together, the source design shows that the project is not only collecting data, but separating `metadata`, `ratings`, `audience text`, `editorial text`, `attention signals`, and `identity alignment` into distinct layers. That distinction is essential both for robust analytics and for later RAG design.

4. How We Can Create a Relational Database Through the Extracted Data

The extracted data supports a clear relational database design because the project deliberately transforms heterogeneous source payloads into a small number of canonical tables linked by explicit keys. The central entity is the `title`. Every movie or TV show is assigned an internal `title_id`, typically derived from the content type and TMDB ID, such as `movie_<tmdb_id>` or `tv_<tmdb_id>`. This internal identifier becomes the stable join key across all downstream tables. The `titles` table is therefore the central dimension table in the system. It stores source-independent title identity together with aligned external identifiers such as `tmdb_id`, `imdb_id`, and `wikidata_id`, along with normalized metadata like title name, original title name, release date, release year, genres, language, country information, overview text, and selected source-specific metadata from IMDb, TVMaze, and Wikidata.

Once the title identity is stabilized, the project organizes text into a separate `documents` table. This is necessary because a title may have multiple different textual artifacts, and those artifacts differ in type, quality, provenance, and intended use. For example, one title may have a TMDB overview, multiple TMDB user reviews, a TVMaze summary, a Wikidata description, and one or more Guardian articles. Storing those directly inside the title row would destroy relational clarity and make later retrieval impossible. Instead, the database models a one-to-many relationship between `titles` and `documents`, where each document keeps its own `document_id`, `source_name`, `document_type`, source URL where available, publication time, author where available, text content, text length, match confidence, and quality flags.

Because the project is also preparing for retrieval, long text documents are further split into a `document_chunks` table. This creates a one-to-many relationship from `documents` to `document_chunks`. Each chunk stores its parent `document_id`, `title_id`, source metadata, chunk index, chunk text, chunk word count, genre, release year, and usability status. This is a relational design decision as much as a retrieval decision. By storing chunks in their own table, later indexing, filtering, and query routing become much easier than if chunking were performed dynamically at query time.

The pipeline also creates a `ratings` table that captures title-level and review-level rating observations from multiple sources. A title can therefore have many associated rating rows, such as a TMDB aggregate rating, an IMDb aggregate rating, and individual review-author scores from TMDB where available. This structure makes it possible to compare platform-level rating signals and track their provenance. In parallel, the project creates an `attention_signals` table, which records Wikipedia

pageviews as time-series events. That table links many daily signals to one title and enables trend analysis over time.

Together, these tables form a clean relational model. The key relationships are `titles` to `documents`, `titles` to `ratings`, `titles` to `attention_signals`, and `documents` to `document_chunks`. This design supports both warehousing and later RAG use because it separates concerns: title identity, text evidence, chunk-level retrieval units, numerical ratings, and temporal attention are stored in different tables but remain joinable through explicit IDs. In other words, the repository is not simply producing flat files; it is producing a relationally coherent analytical dataset.

5. How We Are Preprocessing the Data, What Purpose Each Step Serves

The preprocessing layer in this project is not a cosmetic cleanup stage. It is the core mechanism that turns inconsistent multi-source web and API data into a coherent dataset. The first important preprocessing step is anchor selection. Rather than querying every source independently and trying to merge the results afterward, the project first constructs a master anchor list from TMDB within the configured date window. This ensures that all secondary sources are queried against the same set of target titles. The purpose of this step is to reduce downstream ambiguity and make source alignment explicit instead of accidental.

Once the anchor list exists, the project performs identifier alignment and canonical title construction. The transformation logic reads TMDB details, IMDb matched TSV rows, TVMaze records, and Wikidata payloads, then consolidates them into one canonical title row. This step standardizes identity around an internal `title_id` while preserving external source identifiers. Its purpose is to prevent later joins from depending on fragile string matches and to preserve a reproducible mapping between sources.

Text preprocessing is then applied to document-bearing sources. The repository cleans HTML markup, normalizes whitespace, repairs common encoding problems, and removes malformed characters where possible. The mojibake repair logic is especially important because raw source text can contain broken apostrophes, quote marks, and other Unicode artifacts that would harm readability and downstream retrieval quality. The purpose of this text cleaning step is to produce text that is both human-readable and machine-usable, without losing provenance.

The project also uses explicit source matching heuristics for secondary text sources, especially Guardian and GDELT. Guardian articles are not accepted merely because they were returned by a query. Instead, the code checks title-name occurrences across headline, trail text, body text, and URL/slug patterns, and assigns a confidence score and match method. GDELT is treated even more conservatively, with emphasis on headline matches. The purpose of this logic is to reduce false positives, because incorrect source-title attachment would contaminate both analysis and retrieval.

After document creation, the repository applies quality flagging. Documents can be flagged as empty, too short, weakly matched, or still containing possible encoding noise. These flags are not decorative metadata. They directly influence whether a document is considered usable for future retrieval. The purpose is to keep lower-confidence or low-value text in the dataset for auditability while preventing it from silently entering a future retrieval corpus as trusted evidence.

Deduplication follows. The project removes duplicate documents using source URLs and text hashes. This is necessary because repeated articles or repeated descriptive text can otherwise distort both analytics and retrieval. If one duplicated document appears many times, it can artificially inflate evidence weight, create misleading coverage metrics, and bias future retrieval results. Deduplication therefore serves both storage efficiency and methodological correctness.

The preprocessing layer also handles structured normalization for ratings and attention. Ratings from TMDB and IMDb are reorganized into a single ratings table with explicit source labels, scale ranges, counts, and timestamps where meaningful. Wikipedia pageviews are turned into a daily signal table, but only after verifying that the observed article matches the expected article derived from Wikidata. That validation step protects against one of the most subtle failure modes in cultural datasets: assigning attention to the wrong entity because a title is ambiguous.

Finally, the project prepares the data for future retrieval by chunking the text documents into smaller units. Chunking is performed after cleaning and deduplication, and each chunk inherits metadata such as title, source, genre, release year, match confidence, and usability status. The purpose of chunking is to create retrieval units that are small enough to index and cite later while still preserving enough context to be meaningful. Validation reports are generated at the end so that row counts, missing IDs, duplicates, chunk coverage, and document quality can be inspected systematically. Overall, each preprocessing step exists to serve one or more of four goals: correctness, comparability, provenance, and retrieval readiness.

6. How We Will Load the Data into Datalake/Data Warehouse

The correct architectural distinction in this project is that `Google Cloud Storage` functions as the data lake and `BigQuery` functions as the data warehouse. The repository's cloud publishing layer is built around that separation. The data lake role belongs to GCS because the project preserves extracted and transformed outputs as files. Raw source snapshots, processed JSONL files, CSV exports, manifests, and validation reports are all file-oriented artifacts. They need durable, hierarchical storage that preserves provenance by run ID and allows reprocessing or audit later. That is exactly the role of the data lake in this project.

When the cloud load step is enabled, the repository uploads raw source directories into the raw bucket under a structure such as `raw/<source>/<source_run_id>/...`. This includes directories for anchors, TMDB, IMDb, TVMaze, Wikidata, Wikipedia, Guardian, and optional GDELT. It also uploads processed outputs into the processed bucket under `processed/<process_run_id>/...`, and report artifacts under `reports/<process_run_id>/...`. The purpose of this organization is to preserve both the extraction run and the transformation run as explicit storage partitions. This means that local ETL outputs do not disappear once transformed; they can be re-used, audited, or reloaded later.

BigQuery is used for the warehouse layer because the project's canonical outputs are relational in nature and are expected to support analytical querying. The tables loaded into BigQuery are the main processed tables: `titles`, `documents`, `document_chunks`, `ratings`, and `attention_signals`. The cloud load implementation first ensures that the dataset exists, then checks whether each table already exists. If it does, the loader deletes rows for the same `process_run_id` before appending the new JSONL data. That design makes reloads safer and more reproducible, because each transformation run is uniquely identifiable and can be refreshed without blindly truncating whole tables.

The advantage of this two-layer design is that it separates storage concerns from analytical concerns. GCS keeps raw and semi-processed files exactly as the pipeline produced them, which is important for provenance and replayability. BigQuery, by contrast, stores structured records in a form optimized for SQL joins, metrics, dashboarding, and downstream consumption. For this project, that separation is the correct one. The repository is therefore not simply "uploading files to GCP"; it is implementing a basic but sound data-lake-plus-warehouse architecture.

At the moment, the GCP part is partially operational in code but still pending final environment and permission fixes in practice. The repository already supports the entire sequence, but successful execution depends on the existence of the configured buckets, correct service-account JSON placement, bucket-level object creation permissions, and suitable BigQuery roles. Once those are in place, the loading design is already implemented and aligned with the structure of the ETL outputs.

7. How the Data Is RAG-Ready

The current state of the project is best described as `RAG-ready but not yet a complete RAG system`. The repository already performs several of the most important preparation tasks that a future RAG system would require. It collects text-bearing evidence from multiple sources, cleans and normalizes that text, links each text artifact to a stable canonical title identity, deduplicates repeated content, records provenance such as source name and URL, flags low-quality records, and produces explicit chunk-level outputs. These are exactly the foundational conditions needed before vector indexing and retrieval can be implemented responsibly.

The `documents` table is already a strong retrieval precursor because it contains multiple document types with source metadata, publication information, confidence information, and quality flags. The `document_chunks` table goes one step further by turning those documents into retrieval-sized units. Each chunk keeps a connection back to its parent document and title, while also carrying metadata such as source, content type, release year, genre, and usability. This means the project already produces the kinds of units that a vector store would later index. In addition, the repository preserves run-level provenance through `source_run_id` and `process_run_id`, which is important for reproducible embeddings and index rebuilds later.

However, the project does not yet implement the full downstream RAG stack. There is no embedding generation step, no vector database population, no hybrid retrieval layer, no reranking stage, no prompt orchestration, and no answer generation component. In other words, the repository has completed the `data engineering prerequisites` for RAG but not the `retrieval and generation runtime` itself. That distinction is important for the report, because it would be inaccurate to describe the current system as an operational RAG chatbot.

The most accurate statement is therefore that the repository is already structured to support RAG with relatively little architectural rework. The heavy-lift ETL work needed for future retrieval has already been done: canonical identity resolution, multi-source text collection, cleaning, quality control, deduplication, chunking, and provenance tracking. What remains is the machine-learning and application layer that would consume those outputs. From a project maturity perspective, the data is ready to move into the next phase, but that next phase has not yet been implemented inside this repository.